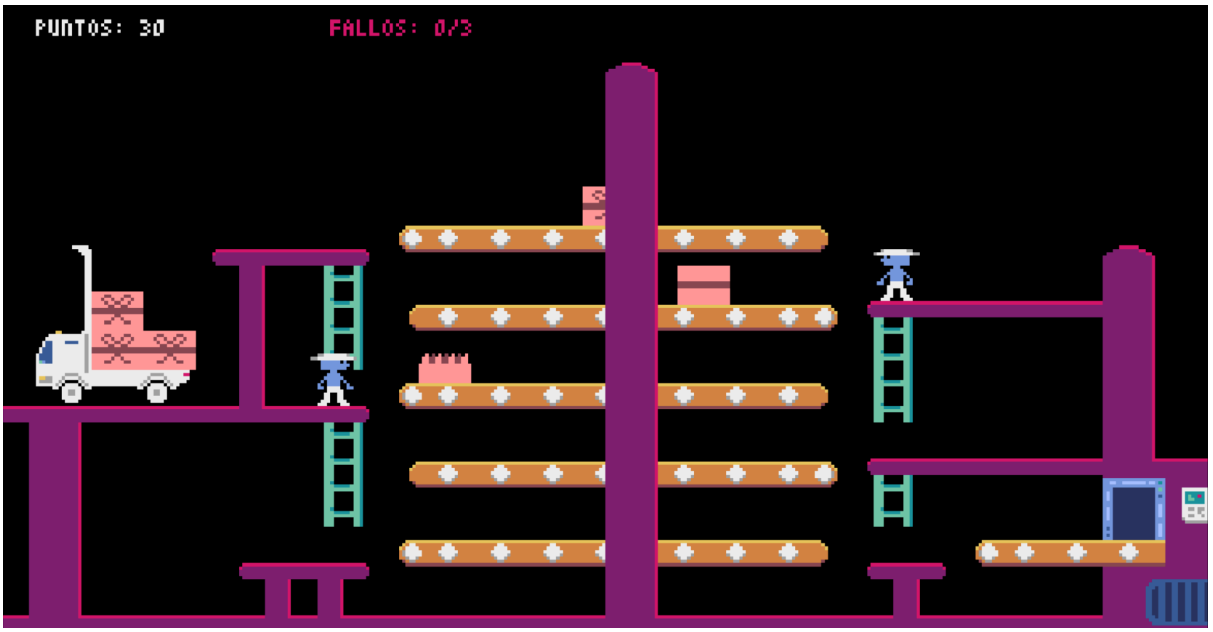
PROYECTO FINAL:

MARIO BROS (Game & Watch)

Asignatura: Programación 2025 - 2026



Autores:

- Juan Gimeno Merino
- Pablo Rojo Castaño

Grupo: 82 **Fecha:** 8/12/2025

ÍNDICE

PROYECTO FINAL: MARIO BROS (Game & Watch)..... 0
Asignatura: Programación 2025 - 2026.....0
ÍNDICE..... 1
RESUMEN.....2
DESCRIPCIÓN DE LAS CLASES.....2
Clase Tablero (Controlador Principal).....2
Clase Personaje..... 2
Clase Cinta..... 3
Clase Paquete..... 3
Clase Camion..... 3
Clase Nivel..... 3
ALGORITMOS.....3
Máquina de Estados Finita (FSM)..... 3
Algoritmo de Movimiento "Zig-Zag" de Paquetes.....4
Sistema de Apilado Visual del Camión.....4
TRABAJO REALIZADO.....4
Funcionalidades Implementadas (Requisitos Obligatorios)..... 4
Funcionalidades Adicionales (Mejoras).....5
Partes No Implementadas..... 5
MANUAL DE USUARIO..... 5
Controles..... 5
Reglas..... 6
CONCLUSIONES.....6

RESUMEN

Este documento detalla el proceso de diseño, implementación y desarrollo de una recreación digital del clásico juego "Mario Bros Game & Watch" utilizando el lenguaje Python y la librería gráfica Pyxel. El proyecto se ha desarrollado siguiendo el paradigma de la Programación Orientada a Objetos (POO), aplicando conceptos clave como encapsulación, herencia y gestión de estados. El informe describe la arquitectura de clases, los algoritmos principales para la simulación de la fábrica y el movimiento de los personajes, así como las funcionalidades extra implementadas.

DESCRIPCIÓN DE LAS CLASES

1. Descripción de las Clases Diseñadas

El proyecto se ha estructurado en diferentes módulos para asegurar un código limpio, modular y escalable. A continuación, se describen las clases principales y sus responsabilidades:

Clase Tablero (Controlador Principal)

Es la clase central que orquesta el juego. Contiene el bucle principal (update y draw) y gestiona la máquina de estados.

- **Atributos principales:**
  - cintas: Lista de objetos Cinta que componen el nivel.
  - mario / luigi: Instancias de la clase Personaje.
  - estado_juego: Variable de control de flujo (MENU, JUGANDO, REPARTO, GAMEOVER).
  - gestor_puntos: Instancia encargada de la persistencia de puntuaciones (Ficheros).
- **Métodos destacados:**
  - configurar_partida(dificultad): Inicializa las coordenadas y velocidades según el nivel elegido.
  - iniciar_reparto(): Activa la secuencia de animación del camión y limpia el tablero.

Clase Personaje

Representa a los protagonistas (Mario y Luigi). Encapsula la lógica de movimiento vertical y la gestión de sprites.

- **Atributos:** piso, x, y, nombre.
- **Métodos:**
  - subir() / bajar(): Modifican el piso lógico y la posición visual asegurando que el personaje aterrice en las plataformas correctas definidas en el Tablero.
  - animar_recogida(): Cambia temporalmente el sprite para dar feedback visual al coger una caja.

Clase Cinta

Modela las cintas transportadoras. Su responsabilidad es mover los paquetes que tiene encima y detectar cuándo llegan al extremo.

- **Atributos:** numero (identificador), paquetes (lista interna de objetos Paquete).
- **Métodos:**
    - actualizar_paquetes(): Aplica velocidad a cada paquete según si la cinta es par (mueve a izquierda) o impar (mueve a derecha).
    - paquete_llego_al_final(): Devuelve el paquete que está listo para ser recogido.

Clase Paquete

Representa las cajas. Posee lógica visual para "evolucionar" su apariencia según avanza por la fábrica.

- **Atributos:** cinta_actual, sprite (propiedad calculada).
- **Métodos:** evolucionar(numero_cinta) cambia el sprite del paquete para dar continuidad visual al pasar de una cinta a otra.

Clase Camion

Gestiona la lógica de recolección final. Implementa un sistema visual de apilado.

- **Atributos:** carga_visual (lista de coordenadas de las cajas apiladas).
- **Métodos:**
    - cargar_paquete(): Calcula la posición (x, y) exacta dentro del remolque para dibujar las cajas apiladas en 2 columnas.
    - mover(): Permite la animación de entrada y salida del camión.

Clase Nivel

Clase de configuración que almacena los parámetros (velocidad, número de cintas, dimensiones) en función de la dificultad ("FACIL", "MEDIO").

ALGORITMOS

2. Descripción General de los Algoritmos Utilizados

Máquina de Estados Finita (FSM)

El juego no es lineal, sino que transita por diferentes estados gestionados en el método update de la clase Tablero.

- **MENU:** Espera input para seleccionar dificultad.
- **JUGANDO:** Ejecuta la lógica principal (movimiento, colisiones).
- **REPARTO:** Bloquea el control del jugador y ejecuta la animación del camión saliendo y entrando.
- **CASTIGO:** Pausa breve donde aparece "The Boss" cuando se cae un paquete.

- **GAMEOVER:** Muestra la puntuación final y gestiona el registro de récords.

Algoritmo de Movimiento "Zig-Zag" de Paquetes

Para simular la fábrica, se implementó una lógica basada en la paridad del número de cinta:

1. Las cintas **PARES** (Derecha) mueven los paquetes decrementando su X (hacia el centro).
2. Las cintas **IMPARES** (Izquierda) mueven los paquetes incrementando su X (hacia el centro).
3. Al llegar al centro, si el personaje correspondiente está en el piso correcto, el paquete se transfiere a la lista de la siguiente cinta ($ID + 1$) y se teletransporta al extremo opuesto, creando el efecto de "subida".

Sistema de Apilado Visual del Camión

Para que las cajas se vean ordenadas dentro del camión, se utiliza un algoritmo matemático simple basado en el índice de carga ($0..7$):

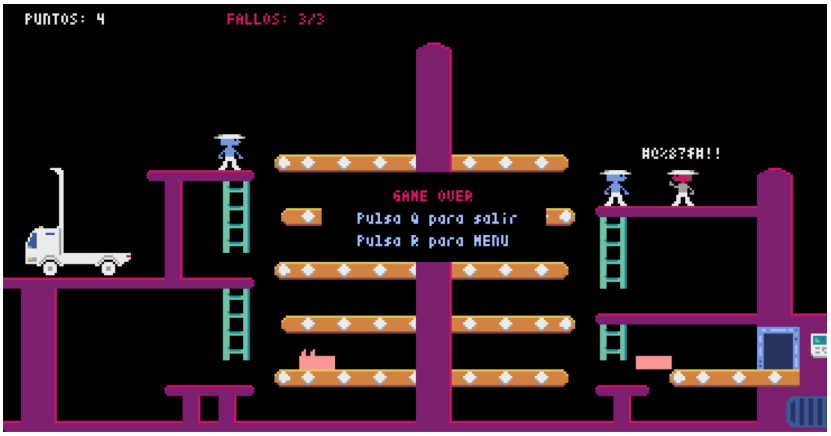
- columna = indice % 2
- fila = indice // 2
  Esto permite calcular el offset exacto (x, y) para dibujar cada caja sin necesidad de crear objetos físicos nuevos, optimizando el rendimiento.

TRABAJO REALIZADO

3. Descripción del Trabajo Realizado

Funcionalidades Implementadas (Requisitos Obligatorios)

- **Movimiento de Personajes:** Control independiente de Mario (Flechas) y Luigi (W/S) con restricciones de piso.
- **Física de Paquetes:** Generación automática, transporte en cintas y gravedad al caer (animación de caída).
- **Sistema de Puntuación:** +1 punto por transferencia, +10 puntos por camión completo.
- **Gestión de Errores:** Contador de fallos con fin de partida al llegar a 3 errores.

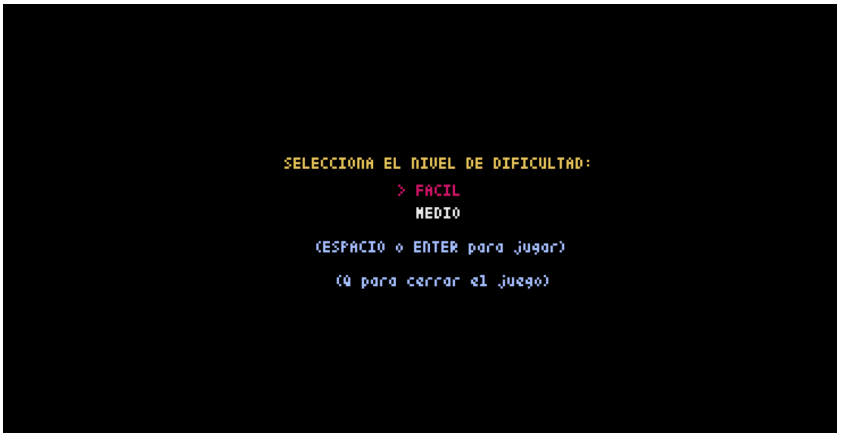


- **Ciclo del Camión:** Lógica de llenado (8 paquetes), animación de salida (descanso) y retorno vacío.
- **El Jefe ("The Boss"):** Aparición con animaciones y mensajes de texto cuando ocurre un fallo o termina el descanso.

Funcionalidades Adicionales (Mejoras)

Para mejorar la experiencia de juego, se han añadido:

1. **Menú de Inicio y dificultad:** Selección entre modo "FÁCIL" y "MEDIO" que altera el número de cintas y la velocidad.



2. **Efectos de sonido:** Implementación de audio retro para acciones (coger caja), eventos (fallo) y melodías (camión, game over) usando el sintetizador de Pyxel.
3. **Animaciones avanzadas:** El camión sale físicamente de la pantalla y vuelve marcha atrás; los paquetes cambian de sprite (abierto/cerrado) según la cinta.

Partes No Implementadas

El juego cumple con el 100% de los requisitos funcionales obligatorios y gran parte de los opcionales. No se han añadido de la parte opcional todos los niveles de dificultad ni el modo competitivo local.

MANUAL DE USUARIO

4. Manual de Usuario

<u>Objetivo</u>

Ayuda a Mario y Luigi a empaquetar botellas en la fábrica. Mario (derecha) debe pasar las cajas a Luigi (izquierda), y este debe subirlas al siguiente nivel hasta llegar al camión de reparto.

Controles

- **Menú:**
  - FLECHAS ARRIBA/ABAJO: Seleccionar opción.
  - ESPACIO / ENTER: Confirmar.
- **Mario (Derecha):**
  - FLECHA ARRIBA: Subir piso.
  - FLECHA ABAJO: Bajar piso.
- **Luigi (Izquierda):**
  - TECLA W: Subir piso.
  - TECLA S: Bajar piso.
- **General:**
  - TECLA Q: Salir del juego.
  - TECLA R: Reiniciar partida (o volver al menú desde récords).

Reglas

1. Sitúa al personaje en el piso correspondiente a la cinta que tiene una caja apunto de caer.
2. Si estás en el sitio correcto, el personaje moverá la caja automáticamente.
3. Si fallas, la caja caerá, recibirás un fallo y aparecerá el Jefe a regañarte.
4. Al completar 8 cajas, el camión saldrá a reparto. ¡Aprovecha para descansar!
5. La partida termina al acumular **3 fallos**.

CONCLUSIONES

5. Conclusiones

El desarrollo de este proyecto ha permitido consolidar los conocimientos de Programación Orientada a Objetos en Python. La implementación de una arquitectura basada en clases independientes (Cinta, Personaje, Tablero) ha facilitado enormemente la gestión de la lógica compleja del juego, como el movimiento simultáneo de múltiples entidades.

Uno de los mayores retos fue la sincronización visual y lógica. Al principio, las coordenadas de los pisos lógicos (0, 1, 2...) no coincidían con los gráficos del archivo .pyxres. Esto se solucionó implementando un sistema de diccionarios de alturas (alturas_mario, alturas_luigi) que mapea la lógica del juego con la posición visual exacta en píxeles.

Otro desafío fue la gestión de estados (pausa durante el reparto), que se resolvió implementando una máquina de estados sencilla en el bucle principal.

El uso de Pyxel ha resultado ser una herramienta excelente para prototipar videojuegos 2D de forma rápida. La inclusión de elementos extra como el sonido ha convertido una práctica académica en un producto de software completo y jugable. Para futuras versiones, se podría implementar un modo de dos jugadores simultáneos en local.